TravelJournal for Mac — Spec

Oct 22, 2026 · @Utsav Pandya

Decisions

TravelJournal becomes a native Mac app built from the same SwiftUI codebase, with iCloud sync so journeys made on iPhone appear on Mac. Calls marked *(Claude)* were made for you because this is your first Mac app; challenge any of them.

Topic	Decision	Why
App type	Native macOS destination in the existing multiplatform target *(Claude)*	Real Mac windows, menus, hover and shortcuts; Catalyst and "Designed for iPad" always feel like a phone app on a Mac
Minimum OS	macOS 27, matching iOS 27 *(Claude)*	Same SwiftUI and SwiftData APIs on both, no availability checks
Journey layout	One continuous horizontal story strip, map pinned on the left	Your choice (option a)
Shelf	Wooden bookcase that reflows with window width; a "+N books" tile reveals the rest	Your choice
Books on shelf	Covers facing out, not spines *(Claude)*	Journeys already have cover photos; spine text is unreadable at small sizes
Sync	iCloud via SwiftData + CloudKit, private database	Your choice; needed so the Mac isn't empty
Editing on Mac	Full editing through a right-side inspector, not sheets *(Claude)*	Mac users edit while seeing the result; a keyboard makes writing stories easier
Opening a journey	Each journey opens in its own window *(Claude)*	Standard Mac behaviour; two journeys side by side
iPad	Out of scope for this phase *(Claude)*	Layout is chosen by available width, so iPad landscape can adopt the strip later with little work

Topic	Decision	Why
PDF	Unchanged: A4 pages on both platforms	The strip is a viewing layout only
Appearance	Light only, same as iPhone	Keeps the Vintage Kraft look consistent

Sharing code across iPhone and Mac

Aim for about 80% shared code: models, sync, page components and services are shared; scenes, navigation and editing chrome are per platform.

Folder structure

- `Shared/Models` — SwiftData models, schema versions, migrations
- `Shared/Story` — page components (ticket, polaroid, tape, stamps, route), layout composers, PDF renderer
- `Shared/DesignSystem` — colour tokens, fonts, textures
- `Shared/Services` — photo import, clustering, place lookup, map snapshots, export
- `iOS/` — iPhone app scene, shelf, swipe viewer, sheets, camera
- `macOS/` — Mac scenes, menus, shelf window, journey window, inspector, drag and drop

Rules

1. **Adapt by space, not by platform.** The journey view picks the strip or the swipe pages from available width, not `#if os(macOS)`. That is what lets iPad landscape use the strip later.
2. **Use `#if os(...)` only for capabilities** that exist on one platform: camera, haptics, menu commands, multiple windows, Settings scene.
3. **One layout engine, two composers.** Keep the page components as they are; add a `StripComposer` next to the existing page composer. Both place the same components, so a fix to a ticket or polaroid fixes both.
4. **Keep images platform-neutral.** Store and pass `Data` or `CGImage`; convert to `UIImage` / `NSImage` only at the edge. A small `PlatformImage` typealias covers the few places that need it.
5. **Share sheet:** use SwiftUI `ShareLink`, which works on both.

Mac gotchas to expect

- **Bundled fonts** are declared differently: iOS uses `UIAppFonts`, macOS uses `ATSApplicationFontsPath` in Info.plist. If Allura or Caveat show as system font on Mac, this is why.
- **Text sizes:** Mac body text is 13 pt versus 17 pt on iPhone. Story pages keep their own fixed sizes (they're designed objects), but app chrome should use system styles so it looks native.
- **Retina and 5K displays:** paper and wood textures need high-resolution assets, or they look soft on large screens.
- **Light only:** apply `.preferredColorScheme(.light)` to every window scene, not just the first.

iCloud sync

Sync uses SwiftData with a CloudKit private database, and it must land before any Mac UI work, because it changes the data model both apps depend on.

Model changes CloudKit requires

- No `@Attribute(.unique)` anywhere.
- Every property is optional or has a default value.
- Every relationship is optional and has an inverse.
- Photos can't stay as files referenced by path, because those files never leave the device. Move photo bytes into the model with `@Attribute(.externalStorage)`, stored as re-encoded JPEG (long edge about 2048 px) to keep iCloud usage reasonable.
- Thumbnails are generated locally on each device and not synced.

Migration

1. Wrap the current models as `SchemaV1` in a `VersionedSchema`.
2. Create `SchemaV2` with the sync-ready changes.
3. A custom migration stage reads each photo file and copies it into the new data attribute, then deletes the file.
4. Remove the silent in-memory fallback first (open item 3 from the v1 status), so a failed migration shows an error instead of an empty shelf.

Configuration

- `ModelConfiguration(cloudKitDatabase: .private("iCloud.<your bundle id>"))`
- Capabilities on both targets: iCloud with CloudKit, the same container, and Background Modes → Remote notifications on iOS.

Sync states the UI must handle

State	What the user sees
Not signed in to iCloud	Shelf works normally, with a small banner: "Journeys stay on this device. Sign in to iCloud to sync."
First launch on a second device	"Bringing in your journeys from iCloud…" instead of the empty-shelf state
Photos still downloading	Polaroid shows a paper-coloured placeholder, then the photo fades in
iCloud storage full	One-time alert explaining new photos won't sync until space is freed

Known limit: conflicting edits generally resolve as last-writer-wins, so editing the same story on two devices at once can lose one version. Acceptable for this phase.

Testing: sync only works on real devices or simulators signed in to an iCloud account, and changes can take seconds to minutes to arrive. Test with an iPhone and the Mac side by side.

Anatomy of a standard Mac app

A Mac app is built from windows, a menu bar, toolbars and a Settings window. Users expect every action to be reachable from the menu bar, with a keyboard shortcut where it's common.

Windows (SwiftUI scenes)

Scene	What it is	Notes
`WindowGroup` — Shelf	The main window, opens at launch	Only one shelf window; reopening the app from the Dock brings it back
`WindowGroup(for: Journey.ID)` — Journey	One window per journey, opened with `openWindow(value:)`	Window title = journey title; reopening an already-open journey focuses its window
`Settings`	Opens with ⌘,	Small, tabbed window: General, Export, iCloud

Windows must remember their size and position between launches, support full screen (⌃⌘F), and have sensible minimums: shelf 720 × 480 pt, journey 1000 × 640 pt.

Menu bar

SwiftUI gives the App, Window and Help menus for free. Add the rest with `.commands`:

- **File** — New Journey, New Journey from Photos…, Add Stop, Import Photos…, Export as PDF…, Print…, Close Window
- **Edit** — Undo, Redo, Cut, Copy, Paste, Delete, Select All, Find (mostly standard, but undo must actually work for stop and leg edits)
- **View** — Show/Hide Sidebar, Show/Hide Inspector, Zoom In, Zoom Out, Actual Size, Enter Full Screen
- **Journey** — Previous Stop, Next Stop, Move Stop Earlier, Move Stop Later, Show on Map, Edit Journey Details
- **Help** — a short help page or link

Menu items grey out when they don't apply (Add Stop with no journey window in front). Use `@FocusedValue` so menus act on the frontmost window's journey.

Toolbar

- Shelf window: New Journey (+), sort menu, search field.
- Journey window: sidebar toggle (left), Edit/Done, Export, Share (right), inspector toggle (far right).
- Use the unified toolbar style so the title and controls share one bar.

Other things Mac users expect

- **Right-click menus** on books, stops, photos and legs.
- **Hover feedback**: pointer changes and subtle highlights on anything clickable.
- **Undo and redo** for every edit (`UndoManager` from the environment; SwiftData can register changes).
- **Printing** with ⌘P: send the A4 PDF pages to the print dialog.
- **Full keyboard use**: Tab moves between controls, Return confirms, Esc cancels or closes popovers.
- **No onboarding carousel**: Mac apps open straight to content with a helpful empty state.

Sidebar, inspector, sheet or popover

On Mac, editing happens beside the content, not on top of it. Sheets are kept for short tasks that must be finished or cancelled before going on.

Rules of thumb

- **Sidebar (left)** — where am I, and where can I go? Navigation and overview.
- **Inspector (right)** — the properties of whatever is selected. Changes apply live.
- **Sheet** — a short, focused task with Cancel and Done (or Create). Blocks the window until finished.
- **Popover** — glanceable info or a tiny edit tied to one spot on screen. Closes on click-away or Esc.
- **Alert / confirmation dialog** — destructive actions only.
- **New window** — something worth keeping open side by side.

Applied to TravelJournal

Situation	iPhone	Mac
Browse journeys	Shelf screen	Shelf window
Open a journey	Push the swipe viewer	New journey window
Map and stop list	Route overview page	Left sidebar, always visible
Edit a stop	Full-screen editor	Inspector (select the stop)
Edit a leg	Leg editor sheet	Inspector (select the leg connector)
Edit journey title, date, intro, closing note	Editor sheet	Inspector (click empty strip or ⌘I)
See leg details	Tap → small sheet	Hover or click → popover
Create a new journey	Sheet	Sheet (title, date, city)
Import photos and confirm stops	Full-screen flow	Large sheet; it's a one-time wizard with a clear end
Delete a stop or journey	Confirmation dialog	Confirmation dialog, also via ⌘⌫
Export PDF	Share sheet	Save panel (`fileExporter`) + Share button in toolbar
View a photo full size	Full-screen cover	Quick Look–style overlay, opened with Space
Settings	Settings screen	Settings window (⌘,)

Situation	iPhone	Mac
Search journeys	Search bar on shelf	Search field in the toolbar (⌘F)

Shelf window

The shelf is a wooden bookcase: journeys stand face-out on wooden planks, and the bookcase rearranges itself as the window is resized.

Look

- Wood-grain shelf planks with a soft shadow under each book; the back wall uses the paper texture.
- Each book shows its cover photo and title in Allura, with a slight random lean (seeded by id, like the story pages).
- Newest journey first.

Reflow

- Books have a fixed size (about 160 × 220 pt). Books per shelf = how many fit the window width; wider window, more books per shelf.
- The number of shelves shown = how many fit the window height.
- Resizing animates books sliding to their new shelf, so it feels physical.

"+N books" tile

- When journeys don't fit, the last slot on the last visible shelf becomes a tile reading "+10 books".
- Clicking it expands the bookcase: the remaining shelves appear and the window scrolls. The tile becomes "Show less" at the end.
- Search and sort always work on all journeys, not just the visible ones.

Search

- Toolbar search field, focused with ⌘F.
- Matches journey title, city, stop names and story text.
- Non-matching books leave the shelf; matching books close the gaps. Empty result: "No journeys match 'temple'."

Interactions

- Double-click or Return opens a journey in its own window.
- Right-click: Open, Open in New Window, Export as PDF…, Print…, Rename, Delete.

- Arrow keys move selection between books.
- Dragging a book out to Finder, the Desktop or Mail exports it as a PDF file (see Drag and drop).
- Dropping photos on the shelf starts "New Journey from Photos".

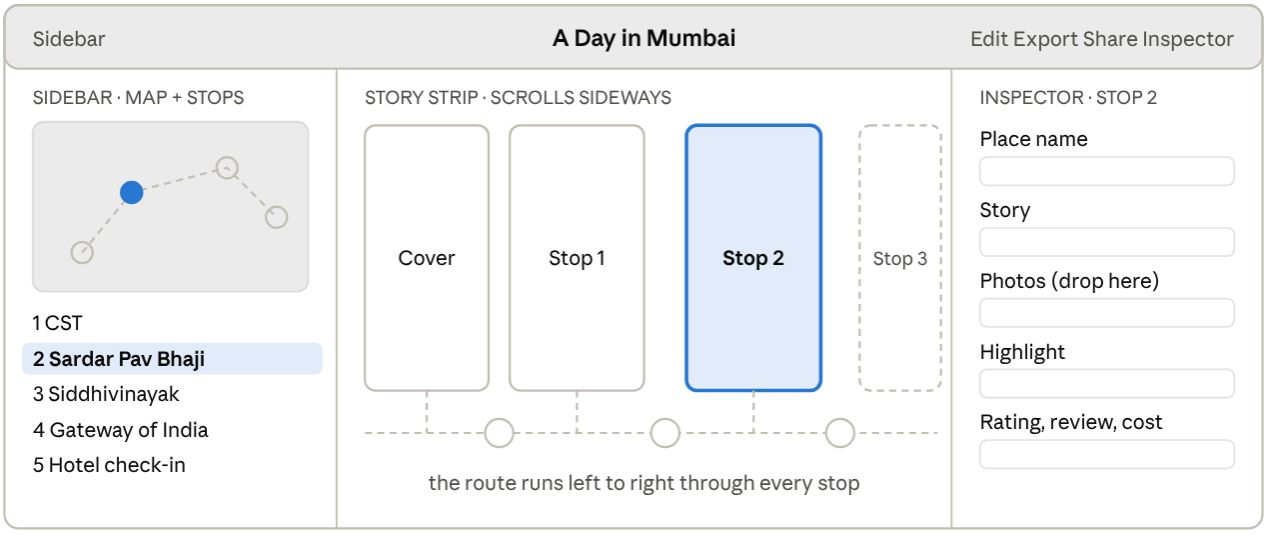
Empty state

An empty bookcase with one line: "Drag photos here, or press ⌘N to start your first journey."

Journey window

The journey window shows the whole day at once: the map and stop list stay pinned on the left, the story flows as one horizontal strip, and the inspector edits whatever is selected.

One window shows the whole journey: map left, story centre, edits right



Hover a stop on the strip: its pin and list row light up. Click a pin or row: the strip scrolls to it.
Hover or click a leg circle: a popover shows mode, km, minutes and the leg story.
Select a stop or leg: the inspector edits it live. Select nothing: it shows the journey details.

Journey window · sidebar, story strip, inspector

Strip behaviour

- Order: cover → stop 1 → leg → stop 2 → … → closing card. No page breaks; the route is one continuous dashed line.
- Built by a new `StripComposer` from the same components as the iPhone pages.
- Trackpad scrolls sideways naturally; a plain mouse wheel's vertical scroll is turned into horizontal movement.
- ← / → jump stop to stop, centring the stop in view.

- ⌘+ / ⌘− zoom the strip; ⌘0 fits the whole journey into the window.
- Sidebar collapses (⌃⌘S) for a wider strip; inspector is hidden until Edit is pressed or ⌥⌘I.
- Sidebar width is resizable between about 220 and 360 pt.

Keyboard shortcuts

Use the shortcuts Mac users already know; only add new ones where nothing standard exists. Shortcuts marked standard come free or nearly free in SwiftUI.

Action	Shortcut	Where	Standard
New Journey	⌘N	Anywhere	Yes
New Journey from Photos…	⇧⌘N	Anywhere	No
Close window	⌘W	Anywhere	Yes
Settings	⌘,	Anywhere	Yes
Search journeys	⌘F	Shelf	Yes
Open selected journey	Return	Shelf	Yes
Export as PDF…	⌘E	Shelf, journey	No
Print…	⌘P	Shelf, journey	Yes
Undo / Redo	⌘Z / ⇧⌘Z	Anywhere	Yes
Delete selected	⌘⌫	Shelf, journey	Yes
Add Stop	⌥⌘N	Journey	No
Previous / next stop	← / →	Journey	No
Move stop earlier / later	⌥⌘↑ / ⌥⌘↓	Journey	No
Show journey details	⌘I	Journey	Yes (Get Info)
Show / hide inspector	⌥⌘I	Journey	Common
Show / hide sidebar	⌃⌘S	Journey	Yes
Zoom in / out / actual size	⌘+ / ⌘− / ⌘0	Journey	Yes
Quick Look selected photo	Space	Journey	Yes
Enter full screen	⌃⌘F	Anywhere	Yes

Action	Shortcut	Where	Standard
Cancel / close popover	Esc	Anywhere	Yes

Arrow keys and Space must not fire while a text field (story, highlight) has focus.

Drag and drop

On Mac, drag and drop is how photos get in and how stories get out; every drop target shows a highlight while something hovers over it.

Drag	Drop on	Result
Photos from Finder or the Photos app	Shelf	Starts New Journey from Photos: clusters into stops, then the confirm-stops sheet
Photos	A stop on the strip, or the inspector's photo well	Adds to that stop; past 4 photos, asks which to keep
Photos	The gap between two stops	Creates a new stop there
A stop in the sidebar list	Another position in the list	Reorders stops; legs re-sync as on iPhone
A photo inside the inspector	Another slot	Reorders photos; first slot is the hero
A book on the shelf	Finder, Desktop, Mail, Messages	Exports the journey as an A4 PDF file
A photo on the strip	Finder or another app	Copies the image file

Implementation notes

- SwiftUI `.dropDestination(for:)` for drops and `.draggable` for drags, built on `Transferable`.
- The journey's PDF export conforms to `Transferable` with a `FileRepresentation`, so dragging a book renders the PDF on demand.
- Photos dropped from Finder have no `PHAsset`, so read GPS and capture time from the image's EXIF with ImageIO (`CGImageSource`). Photos dragged from the Photos app usually arrive as files with EXIF too; test both.
- Accept HEIC, JPEG and PNG; ignore other file types silently with a brief "Only photos can be added" hint.

Sandbox, entitlements and release

Mac App Store apps run in the App Sandbox, so every capability the app uses must be declared; a missing entitlement fails silently or crashes only on Mac.

Entitlement	Needed for
App Sandbox	Required for the Mac App Store
User Selected File: Read/Write	Save panel for PDF export, files dragged in from Finder
Outgoing Connections (Client)	MapKit search and snapshots, CloudKit
Photos Library	Reading photo metadata (location, date) beyond what PhotosPicker hands over
iCloud → CloudKit, same container as iOS	Sync
Printing	⌘P

The camera and location entitlements aren't needed on Mac, since capture stays iPhone-only.

Release

- Same bundle ID for iOS and macOS in the multiplatform target gives Universal Purchase: buy once, get both.
- TestFlight works for Mac builds; testers install through the TestFlight Mac app.
- Mac App Store review checks that menu items work, windows resize sensibly, and nothing is a stub. Hide unfinished menu items rather than leaving them disabled forever.
- Mac screenshots for the store are 16:10 (e.g. 2880 × 1800).

Build order and test checklist

Build in this order; each phase leaves both apps working, so iPhone never breaks while Mac is in progress.

1. **Sync-ready data:** remove the in-memory fallback, add `SchemaV1` / `SchemaV2`, migrate photos into external storage, turn on CloudKit. Ship to iPhone first and confirm existing journeys survive the migration.
2. **Mac compiles:** add the macOS destination, folder split, `PlatformImage`, Mac font registration, entitlements. Goal: the iPhone UI runs in a Mac window with no crashes.

3. **Scenes and menus:** shelf window, journey windows, Settings, menu bar commands, toolbar.
4. **Shelf:** wooden bookcase, reflow, "+N books", search, sort, context menu, empty state.
5. **Journey window:** sidebar map and stop list, `StripComposer`, hover and click linking, leg popovers, keyboard navigation, zoom.
6. **Editing:** inspector for journey, stop and leg; undo/redo; import-photos sheet.
7. **Drag and drop:** all rows in the Drag and drop table.
8. **Export and print:** Save panel, Share, ⌘P, drag-out PDF.
9. **Polish and TestFlight.**

Test checklist

- [ ] Existing iPhone journeys migrate without losing photos
- [ ] A journey made on iPhone appears on Mac, and an edit on Mac reaches iPhone
- [ ] App works with iCloud signed out, and shows the banner
- [ ] Windows at minimum size, half screen, full screen and on a 5K display
- [ ] Trackpad horizontal scroll and a plain mouse wheel both move the strip
- [ ] Every menu item works or is greyed out correctly for the front window
- [ ] Every shortcut in the table works, and none fire while typing a story
- [ ] Photos dropped from Finder and from the Photos app keep their GPS and time
- [ ] Dragging a book to the Desktop produces the same PDF as Export
- [ ] Undo restores a deleted stop with its photos and legs
- [ ] Fonts render as Allura, Lora, Caveat and Special Elite, not system fonts

Open questions

- [ ] Should the wooden bookcase also replace the iPhone shelf, for one consistent look?
- [ ] Book covers facing out, or spines (more realistic, harder to read)?
- [ ] Bring iPad into scope now, given iPad landscape could reuse the strip layout?
- [ ] Should the Mac strip print as a long panorama poster in addition to A4 pages?